

Retrospective Sprint 2 of Group UG13

Block Model Compression Algorithm

Team Members:

1. a1837374 Mengbo Ren
2. a1837306 Suchang Liu
3. a1824243 Yuyao Chen
4. a1837254 Yang Shi
5. a1832623 Hin Kai Chan
6. a1860970 Mohsan Salehi-Pugsley
7. a1818124 Chih-Hao Chang

Snapshots

I attended the sprint review/planning meeting on 21/Aug with the tutor.

Snapshots are provided in the appendix at the end of the retrospective.

What went well in the sprint?

Our primary objective of the second sprint is to deliver tangible compression software. Some success achieved includes adopting the methodology to research, then develop, and finally test. The emphasis on initial research promoted us to leverage from the existing compress algorithms and thus, reduced the trials and errors of starting from scratch, saving us time and effort.

Additionally, the team members worked in parallel for each algorithm, which raises productivity (by decreasing the idle time for dependent tasks). For instance, two members could research together on a given algorithm, with one implementing it, and the other constructing test cases, analyzing complexity, and documenting.

Besides that, the team also worked in parallel on different algorithms; at any point in time, there would be different members assigned to work on different algorithms. This way of task allocation has several benefits for the initial development phase. First, code reuse across different algorithms is achieved. For example, most basic algorithms utilize Run Length Encoding (RLE) and a rolling window. These functionalities are reusable within the team to increase the productivity of development. Moreover, oversights may be caught with peer code reviews during reuse. More importantly, algorithms that are more appropriate to the project may be chosen with the help of comparative insights from diversified algorithms. For instance, team members could show how one algorithm is more suitable when compared with other algorithms, thereby enabling informed decisions.

What could be improved?

1. Task Complexity and Dependency Analysis

The team should focus on more thorough task complexity and dependency analysis so that the projects can be efficient and thus satisfactory for the clients. In this sprint, we underestimated the complexity of implementing 3D compression for the third dimension of the input due to limitations in the implementation of the input (unable to fulfill the need for trace-back). This shows the importance of detailed task complexity and dependency analysis. In addition, introducing detailed time-bound estimates of tasks can help to improve resource allocation, schedule planning, and communication with the proxy client.

2. Progress Tracking

Further, GitHub issues should be maintained more frequently for transparency and problem resolution purposes, detailing task progress updates. During this sprint, discussions within the group came up with the solution, but it had not been documented in detail. This instance of not being able to present information quickly during the client meeting emphasizes the need for comprehensive documentation. In addition, detailed progress tracking facilitates the timely identification of bottlenecks and resource allocation issues. For example, in some cases where the task was more complex than it has been estimated, the team leader can assign more team members to fulfill the task on time.

3. Product Backlog Refinement

Finally, elaboration is needed on product backlog refinement by having more user stories, so that the client can gain a quicker and a more in-depth insight on how we will proceed through the upcoming sprints and whether or not it matches their expectations of outcome.

What will the group commit to improve in the next sprint?

We are committed to making some improvements to our project management process based on our clients' suggestions. These include improvements in:

1. Task Analysis and Time Management

Learning from the mistakes mentioned previously, the team is committed to dedicating more time to the analysis of tasks during the scrum meetings. This will help us estimate task complexity and dependencies more effectively and thereby, define better time bounds and resource allocation. To that end, detailed task analysis will be carried out in future sprint planning meetings.

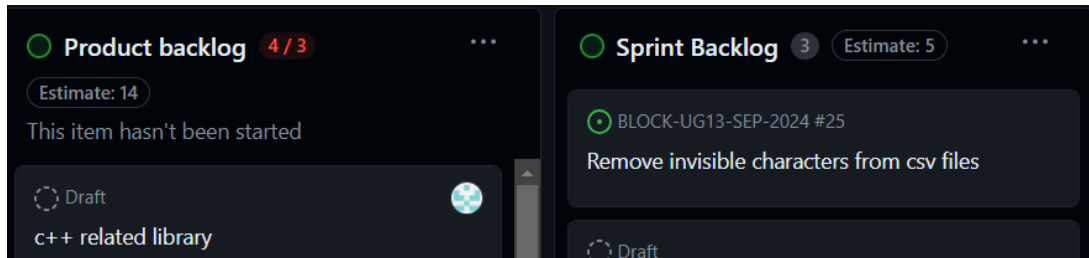
2. Task Progress Documentation

The team agreed upon updating the task progress on GitHub with a more comprehensive description. To explain further, when any member finalizes a task, he needs to write a summary describing the problem and approach details for the solution. In addition, to ensure comprehensive documentation, one member will take the responsibility to document the group's solutions for a specific task, rather than avoiding relying on individual notes. This will help the client understand the key features' solutions of our team, as well as provide a platform for easy reference of similar problems in the future.

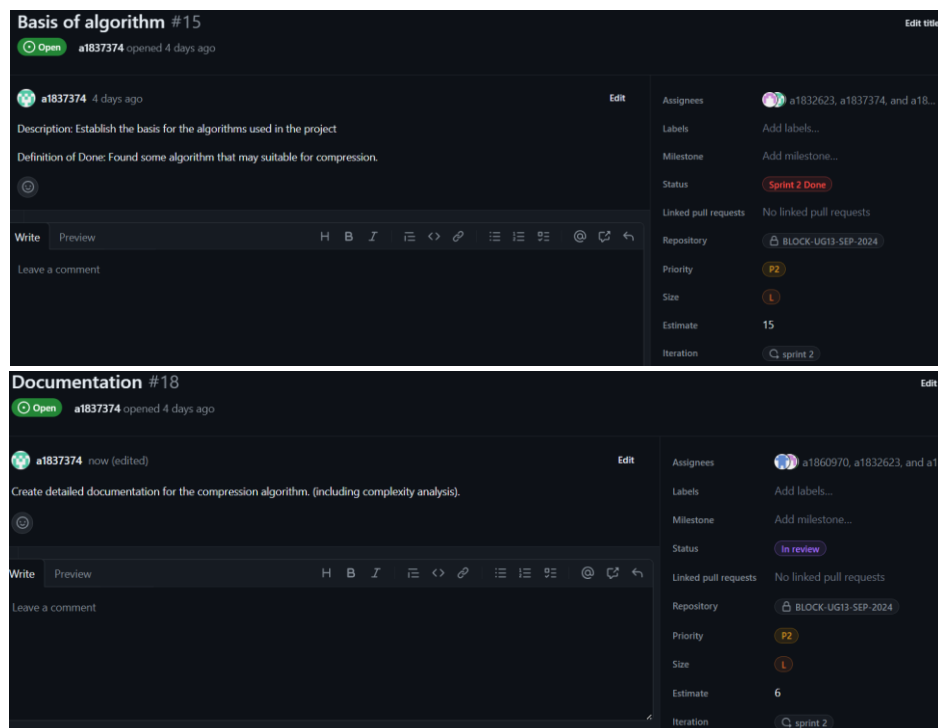
3. Product Backlog

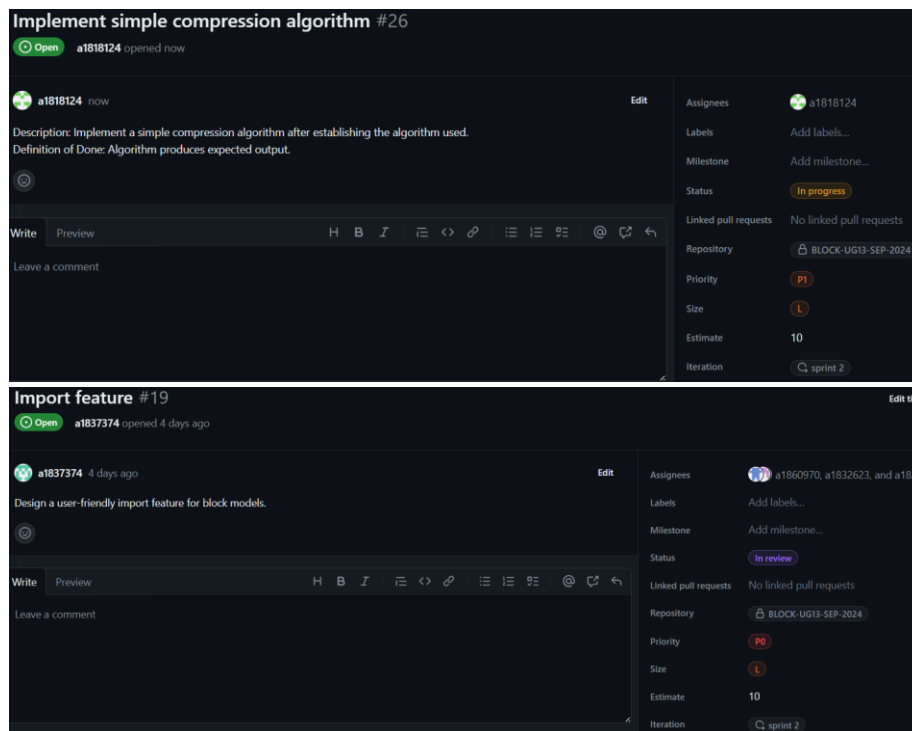
We decided to utilize the product backlog for storing tasks that are not scheduled into the current sprint backlog. These will include ideas on algorithm design, bottlenecks to be discussed in team meetings, and draft user stories for future sprints. The product backlog will be helpful during the sprint planning meetings as the team can have an easily assessable and comprehensive list of user stories to

pick from. For example, if a task is tightly coupled with multiple tasks and has not been addressed, dragging it from the product backlog to the sprint backlog for prioritization in the next sprint would be easy. Also, these detailed backlogs will give the client a better idea about our project planning and subsequent development.



My progress this sprint





During the second sprint, I focused on the research and development of a basic compression algorithm. This will entail researching a variety of compression algorithms, documenting our various approaches along with analyzing the complexity, and implementing a new method of block creation for comparison.

1. Research and Selection of Algorithm

I researched some available compression algorithms and found the run-length encoding algorithm (RLE) to be one of the most common lossless data compression algorithm. RLE works great in compressing repeated data; however, it alone was not sufficient to build the required rectangular blocks of equivalent inputs for our project. After a further search, we found that the Connected Component Labeling (CCL) algorithm could be used to create these blocks after applying modifications.

2. Algorithm Development and Implementation

We implemented a simple compression algorithm, merging ideas of RLE and CCL with modification and additional features such as rolling window to suit the project needs. We also wrote research summary and complexity analysis for reference by other team members.

3. Alternative Approach

In the process of doing this research, I elaborated a simpler block formation approach based on the point coordinates. It does not use CCL, which examines and expands each input element's block, but involves the collection of pairs of adjacent points and the collation of blocks based on the rectangular arrangement of these points.

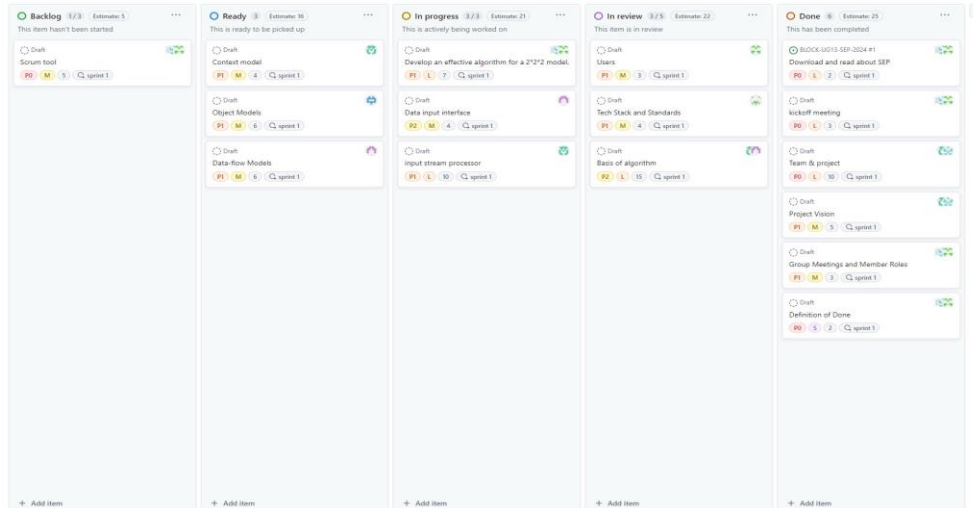
4. Trade-offs between Approaches

However, the discussion in the group suggested that this alternative approach is very memory-intensive for big inputs with many scattered pairs of connected components. Given such a limitation, we focus on the initial approach for the compression algorithm to timely achieve the goal of the second sprint.

Appendix

First Snapshot

Product Backlog and Task Board:



Sprint Backlog and User Stories:

Sprint Overview

For the current sprint, we have selected the following user stories to focus on, catering to both developers and clients:

1. As a developer, I want to have a clear and concise API documentation so that I can quickly understand and implement the block model compression algorithm.

- Task 1.1: Create detailed API documentation for the compression algorithm.
- Task 1.2: Ensure documentation covers all public methods and parameters.
- Task 1.3: Provide code examples for common use cases.

2. As a client, I want to be able to easily import my block models into the program so that I can begin the compression process without any hassle.

- Task 2.1: Design a user-friendly import feature for block models.
- Task 2.2: Ensure compatibility with various block model formats.
- Task 2.3: Implement error handling for import issues.

These user stories and their related tasks will guide our development efforts for the current sprint, ensuring that we address the needs of both developers and clients in creating an effective block model compression program.

ID	Title	Description	Priority	Estimated Effort	Responsible
SB-001	Scrum tool	Register, try, compare and choose Scrum management tools	2	2	
SB-002	API documentation	Create detailed API documentation for the compression algorithm.	1	3	
SB-003	Methods & parameters	Ensure documentation covers all public methods and parameters.	1	4	
SB-004	Code examples	Provide code examples for common use cases.	2	5	
SB-005	Import feature	Design a user-friendly import feature for block models.	1	3	
SB-006	Compatibility	Ensure compatibility with various block model formats.	2	4	
SB-007	Error handling	Implement error handling for import issues.	2	5	

Definition of Done

Users can input an array of 3D block data into the program and expect the output to compress the input data. Currently, the expected input parent block size is 2*2*2.

1 Code complete:

All planned functionality has been implemented, and the code conforms to the team's coding specifications.

2 Code Review passed:

The code has been peer reviewed at least once and all issues raised have been addressed.

3 Compile pass:

Compile through, there are no errors reported, and there are no warnings that affect the operation.

4 Tests passed:

All relevant automated tests (e.g., unit tests, integration tests) pass, and test coverage meets the standards agreed upon by the team.

All relevant manual tests (e.g., functional tests, acceptance tests) passed and no new defects were found.

5 Performance tests passed:

If applicable, all relevant performance tests have been passed and the performance metrics meet the standards agreed upon by the team.

6 Documentation updates:

All relevant technical documentation (such as design documentation, API documentation) has been updated and is consistent with the code.

7 Code merged into the main branch:

The code has been merged into the main branch and no new conflicts or problems have been introduced.

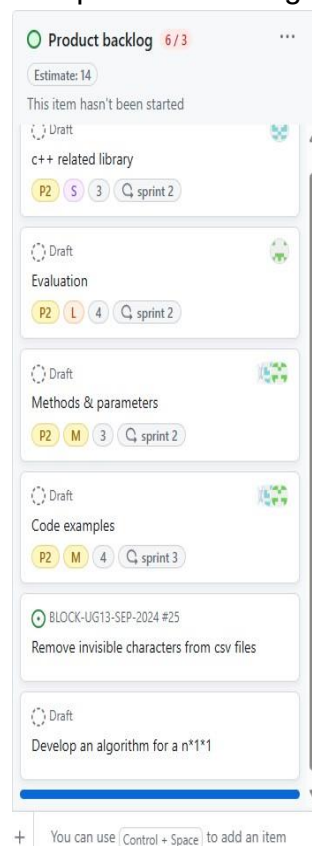
Summary of Changes:

This week's snapshot mainly focused on tracking our latest progress on the backlog and task board. For example, we have identified some viable algorithms that are currently in the "review" phase, where everyone discussed the pros and cons of these algorithms. We also decided on the schedule for all meetings and assigned responsibilities to each team member. Additionally, we discussed our goals and began implementing the input stream processor and input interface, which are now marked as "in progress."

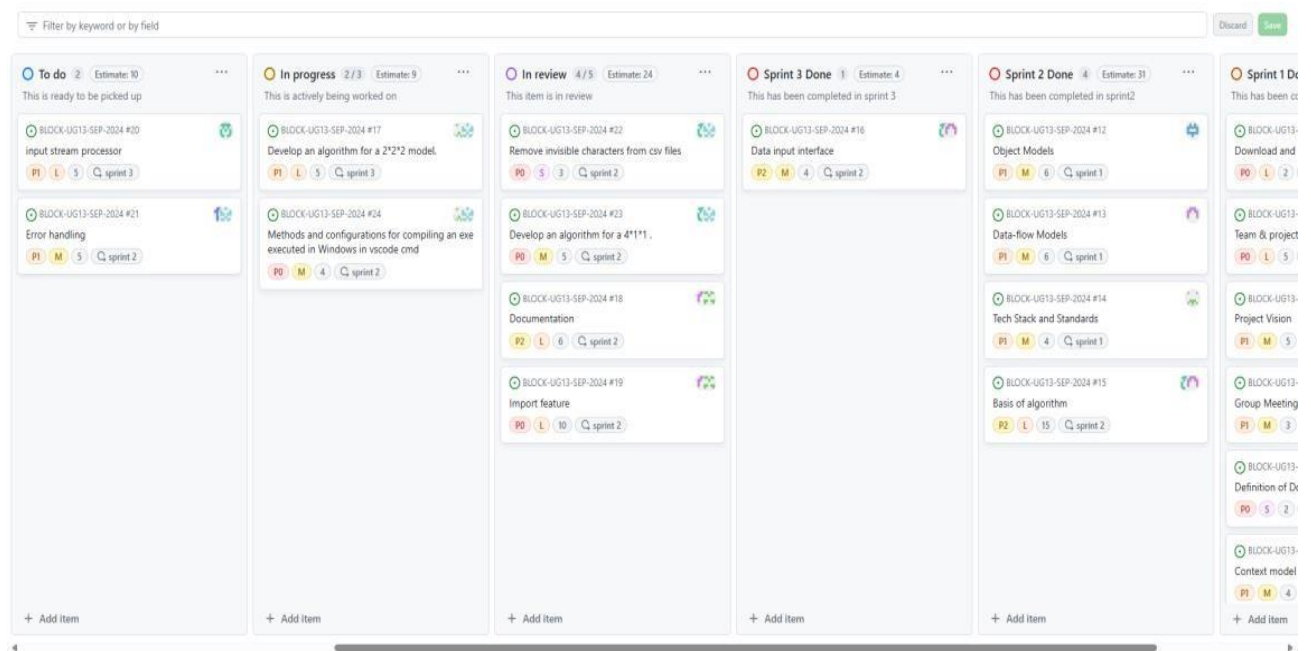
Second Snapshot

Product Backlog and Task Board:

The product backlog



The task board



Sprint Backlog and User Stories:

Sprint backlog



user stories

Sprint Overview

For the current sprint, we have selected the following user stories to focus on :

1. As a developer, looking up `line.empty()` during debugging results in an error.

Although the csv file looks like a blank line, it always doesn't work when determining conditions, resulting in errors.

2. As a developer, in order to familiarize yourself with the testing process, first implement a simple full version with compression capabilities in one direction.
3. Optimize data entry based on the results of previous sprints. A. File data.CVS input and stream; B. Check whether the printed data is consistent with the input data. C. By increasing the amount of data in the csv document, the test can run properly in stream mode.

These user stories and their related tasks will guide our development efforts for the current sprint, ensuring that we address the needs of both developers and clients in creating an effective block model compression program.

Definition of Done

1 Code complete:

Complete the code for the above three user use cases 2 Tests passed:

The code passed the local test, conforming to logic and the output met expectations.

Summary of Changes:

1. Modified the form of the runtime data file name as a parameter;
2. Checked and deleted the characters that could not be printed and displayed in the data file, and corrected the process error;
3. change the compilation environment cmd, and install MinGW compiler, so that exe can run normally under windows